

Pipeline para un Blockchain Artesanal

Hash + Firma Digital + Encadenamiento + Time Stamping

José de Jesús Angel Angel

8 de septiembre de 2026

Alcance de este pipeline

Construimos, con los ingredientes mínimos, una cadena de bloques artesanal capaz de registrar transacciones del tipo **"Bob envía X pesos a Alice, firmado por Bob"**. Solo usamos cuatro primitivas: una **función hash**, una **firma digital**, un **encadenamiento** de bloques y un **sellado de tiempo**. Sin minería, sin consenso distribuido, sin red P2P: el objetivo es entender el esqueleto criptográfico.

1 Las cuatro piezas

1. **Función hash** $H(\cdot)$: comprime cualquier dato a una huella fija (ej. SHA-256, 256 bits). Cualquier cambio mínimo en la entrada altera completamente la salida (efecto avalancha).
2. **Firma digital**: par de claves (sk_B, pk_B) de Bob. Bob firma con sk_B ; cualquiera verifica con pk_B . Prueba autoría y evita que Bob niegue la transacción (no repudio).
3. **Encadenamiento**: cada bloque incluye el hash del bloque anterior, formando una lista enlazada criptográficamente. Alterar un bloque rompe todos los hashes posteriores.
4. **Time stamping**: se agrega una marca de tiempo t a cada bloque, fijando su orden temporal de forma verificable.

2 Estructura de un bloque

$$B_n = (\text{idx}_n, t_n, \text{tx}_n, h_{n-1}, \sigma_n), \quad h_n = H(\text{idx}_n \parallel t_n \parallel \text{tx}_n \parallel h_{n-1}) \quad (1)$$

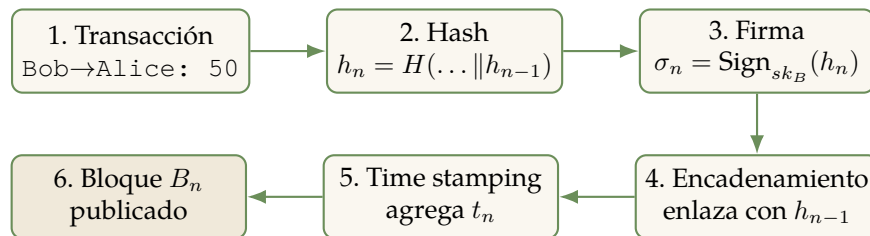
donde:

- idx_n : número de bloque.
- t_n : sello de tiempo (timestamp) del bloque.
- tx_n : la transacción, ej. "Bob envía 50 a Alice".
- h_{n-1} : hash del bloque anterior (encadenamiento).
- $\sigma_n = \text{Sign}_{sk_B}(h_n)$: firma digital de Bob sobre el hash del bloque.

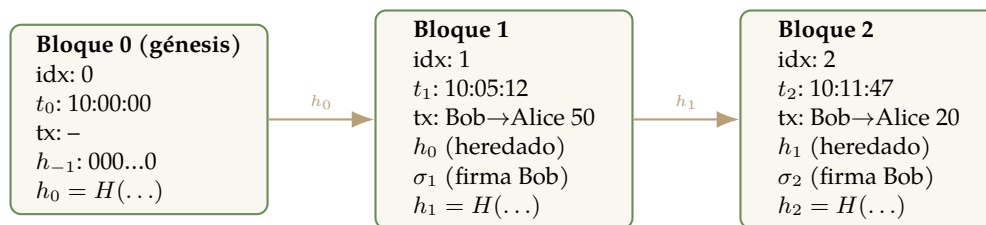
- h_n : hash propio del bloque, que se usará como h anterior del siguiente bloque.

El bloque génesis ($n = 0$) no tiene bloque anterior, así que se define por convención $h_{-1} = 0 \dots 0$ (una cadena fija de ceros).

3 Diagrama del pipeline



4 Encadenamiento de bloques



Si alguien modifica la transacción del Bloque 1 (por ejemplo, cambia “50” por “500”), h_1 cambia por completo (avalancha), la firma σ_1 deja de verificar contra el nuevo h_1 , y el hash heredado en el Bloque 2 ya no coincide. La cadena queda visiblemente rota.

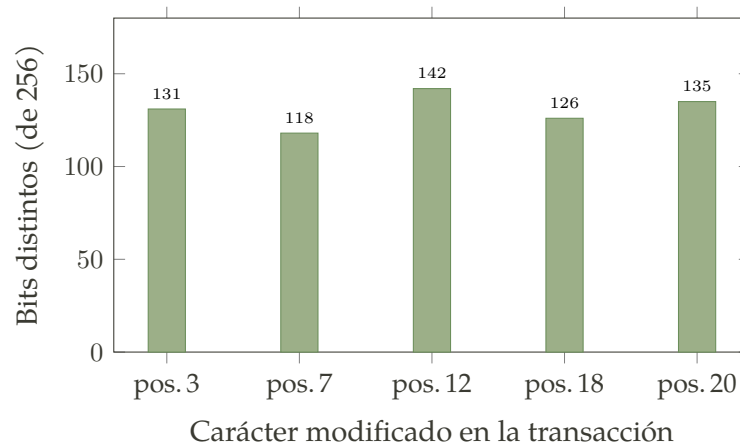
5 Efecto avalancha: ejemplo corto

Un cambio de un solo carácter en la transacción produce un hash completamente distinto (ejemplo ilustrativo con SHA-256 truncado a 16 caracteres hexadecimales):

Entrada	Hash (ejemplo, truncado)
"Bob envía 50 a Alice"	3f2a9c1e7d804b6a...
"Bob envía 500 a Alice"	91e0447bfa2c18d5...

Los valores anteriores son ilustrativos (no recalculados en este documento); lo importante es la propiedad: una diferencia mínima en la entrada produce salidas hash sin ninguna relación aparente entre sí.

5.1 Gráfica ilustrativa: bits que cambian por carácter alterado



Datos simulados para ilustrar la idea: en un hash criptográfico bien diseñado, se espera que en promedio cambie cerca de la mitad de los bits de salida (≈ 128 de 256), sin importar qué carácter se modifique.

6 Pseudocódigo del pipeline (ejemplo corto)

Listing 1: Construcción artesanal de un bloque

```
1 import hashlib, time
2
3 def hash_bloque(idx, t, tx, hash_anterior):
4     contenido = f"{idx}{t}{tx}{hash_anterior}".encode()
5     return hashlib.sha256(contenido).hexdigest()
6
7 def crear_bloque(idx, tx, hash_anterior, clave_privada_bob):
8     t = time.time() # time stamping
9     h = hash_bloque(idx, t, tx, hash_anterior) # hash + encadenamiento
10    firma = firmar(clave_privada_bob, h) # firma digital de Bob
11    return {"idx": idx, "t": t, "tx": tx,
12            "hash_anterior": hash_anterior,
13            "hash": h, "firma": firma}
14
15 # Ejemplo: Bob envia 50 pesos a Alice
16 genesis = crear_bloque(0, "GENESIS", "0"*64, clave_bob)
17 b1 = crear_bloque(1, "Bob envia 50 a Alice",
18                  genesis["hash"], clave_bob)
```

Listing 2: Verificación de la cadena

```
1 def verificar_cadena(bloques, clave_publica_bob):
2     for i, b in enumerate(bloques):
3         h_esperado = hash_bloque(b["idx"], b["t"], b["tx"],
4                                   b["hash_anterior"])
5         assert h_esperado == b["hash"], "Hash alterado"
6         assert verificar_firma(clave_publica_bob,
```

```

7         b["hash"], b["firma"])), "Firma invalida"
8     if i > 0:
9         assert b["hash_anterior"] == bloques[i-1]["hash"], \
10             "Cadena rota"
11     return True

```

7 Pipeline resumido

Etapa	Entrada	Operación	Salida
Transacción	Bob, Alice, monto	Formatear tx	tx_n
Hash	tx_n, h_{n-1}, t_n	$H(\cdot)$ SHA-256	h_n
Firma	h_n, sk_{Bob}	Firma digital (ej. ECD-SA)	σ_n
Encadenamiento	h_{n-1}	Referencia al bloque previo	enlace
Time stamping	reloj local/NTP	Captura t_n	t_n
Bloque final	$idx, t_n, tx, h_{n-1}, \sigma_n, h_n$	Ensamblar y publicar	B_n

Siguiente nivel (fuera de este alcance)

Este pipeline artesanal registra una sola transacción por bloque y confía en que Bob controla su propia cadena. Para pasar a un blockchain real haría falta: un mecanismo de consenso (PoW/PoS) para múltiples nodos, una red P2P para propagar bloques, y una regla de resolución de conflictos (cadena más larga/pesada) cuando dos nodos generan bloques distintos al mismo tiempo.